SMART BOOK SHOP

Online Bookshop & Stationery Management System

Project Report

Object Oriented Programming

CSC 2032

AS20240495 – H.Y. SEWMINI

AS20240580 – P.W.P.R. BANDARA

Git Repo: https://github.com/PamidiBandara/OOP-Project

Submitted Date: 26/07/2026

Table of Contents

1. Introduction

1.1 Purpose

This document is a Software Requirements Specification (SRS) for Smart Book Shop, a full-stack e-commerce web application for the online sale of books, stationery and study kits. It specifies the functional and non-functional requirements, system architecture, external interfaces and design constraints of the system for use by developers, testers, reviewers and other stakeholders.

1.2 Document Conventions

Requirements are labelled using the pattern FR-<module>-<number> for functional requirements and NFR-<number> for non-functional requirements. The keywords “shall”, “should” and “may” are used respectively to denote mandatory, recommended and optional requirements, consistent with common SRS practice.

1.3 Intended Audience and Reading Suggestions

This document is intended for the development team, project reviewers/examiners, quality assurance engineers and future maintainers of the system. Section 2 gives a high-level overview suited to first-time readers; Sections 3-5 give detailed requirements suited to developers and testers; Section 6 documents the internal object-oriented design suited to code reviewers.

1.4 Project Scope

Smart Book Shop provides a public storefront for browsing and purchasing products, a customer account area for order and profile management, and an administrative back office for managing catalogue, orders and customers. The system supports two primary user roles — CUSTOMER and ADMIN. Customers can browse products, manage a cart and wishlist, place orders, write reviews and track deliveries. Administrators can manage the product catalogue, categories, orders and customers through a dedicated admin dashboard. The backend exposes over 65 REST API endpoints across 9 functional modules.

The project also serves as a demonstration of Object-Oriented Programming (OOP) principles applied throughout the backend design; the resulting design decisions are documented in Section 6.

1.5 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications
- Spring Boot 3 / Spring Framework documentation
- MongoDB / Spring Data MongoDB documentation
- React 19 documentation

2. Overall Description

2.1 Product Perspective

Smart Book Shop is a new, self-contained full-stack system composed of a React single-page application (client), a Spring Boot REST API (server) and a MongoDB document database. The backend follows a layered (n-tier) architecture in which each layer communicates with adjacent layers only through public interfaces:

- Controller Layer — 9 REST controllers that accept HTTP requests and delegate to services.
- Service Layer — business logic, expressed as 9 interfaces with separate implementation classes (7+), enabling abstraction and polymorphism.
- Repository Layer — 17 Spring Data repository interfaces that abstract MongoDB persistence.
- Security Layer — JWT filter and Spring Security configuration protecting endpoints by role.

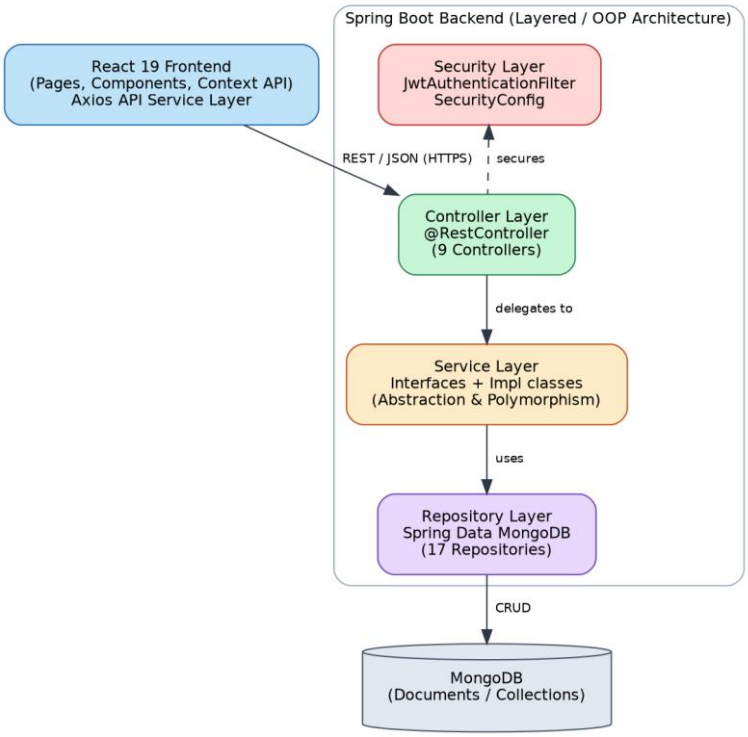


Figure 1: Three-tier / layered system architecture

2.2 Product Functions

At a high level, the system shall provide the following groups of functions (detailed in Section 3):

- Account registration, authentication and profile management
- Product catalogue browsing, search and filtering
- Shopping cart and wishlist management
- Order placement, tracking, history and cancellation
- Product reviews and ratings
- Notifications and recently-viewed products
- Administrative management of products, categories, orders and customers

2.3 User Classes and Characteristics

User Class	Description
Customer	A registered end user who browses the storefront, manages a cart/wishlist, places and tracks orders, and writes reviews. Requires no technical expertise.
Administrator	A staff user with elevated privileges who manages the product catalogue, categories, orders and customers via an admin dashboard. Assumed to have basic familiarity with back-office web applications.
Guest (unauthenticated visitor)	May browse the public storefront and view products, but must register/login to purchase.

2.4 Operating Environment

Layer	Technology
Frontend	React 19, React Router, Vite, Tailwind CSS, Axios, Context API
Backend	Java 17, Spring Boot 3, Spring Web (REST), Spring Security, Lombok
Database	MongoDB (Spring Data MongoDB)
Authentication	JSON Web Tokens (JWT), BCrypt password hashing
Build Tools	Maven (backend), npm / Vite (frontend)
File Storage	Cloudinary (product images)

2.5 Design and Implementation Constraints

The backend must be implemented in Java as an object-oriented codebase applying the four pillars of OOP (Encapsulation, Abstraction, Inheritance, Polymorphism) together with Composition and Aggregation relationships between domain entities. These constraints, and how they are satisfied in the implementation, are documented in Section 6 (System Design Notes).

2.6 Assumptions and Dependencies

- The system assumes availability of a MongoDB instance and network access to the Cloudinary image-hosting service.
- It is assumed that clients interact with the backend exclusively via the documented REST API (Section 4.2).
- Development assumes Java 17 and Node.js/npm toolchains are available in the build environment.

3. System Features (Functional Requirements)

This section enumerates the functional requirements of the system, grouped by module. Each requirement shall be verifiable through testing.

3.1 Authentication & Account Management

ID	Requirement
FR-AUTH-1	The system shall allow a new user to register with a name, email and password, storing the password using BCrypt hashing (never in plain text).
FR-AUTH-2	The system shall allow a registered user to log in with email and password and shall issue a signed JSON Web Token (JWT) on success.
FR-AUTH-3	The system shall reject requests to protected endpoints that do not present a valid JWT.
FR-AUTH-4	The system shall enforce role-based access control, restricting ADMIN-only endpoints to users with the ADMIN role.
FR-AUTH-5	The system shall allow a customer to view and update their own profile information.

3.2 Product Catalogue

ID	Requirement
FR-CAT-1	The system shall allow any visitor to browse the product catalogue, organised into categories (Books, Stationery, Study Kits).
FR-CAT-2	The system shall allow searching and filtering of products by keyword, category and other attributes, with paginated results.
FR-CAT-3	The system shall display a list of featured products on the storefront.
FR-CAT-4	The system shall record and display recently-viewed products for a logged-in customer.

3.3 Cart and Wishlist

ID	Requirement
FR-CART-1	The system shall allow a customer to add, update the quantity of, and remove products in a shopping cart.
FR-CART-2	The system shall allow a customer to add or remove products from a wishlist.
FR-CART-3	The system shall persist cart and wishlist contents against the customer's account.

3.4 Order Management

ID	Requirement
FR-ORD-1	The system shall allow a customer to check out the cart, supplying a delivery address, to place an order.
FR-ORD-2	The system shall record each order as a composition of line items (Order.OrderItem) and a single tracking sub-object (Order.OrderTracking).
FR-ORD-3	The system shall allow a customer to view order history and the current status of any order.
FR-ORD-4	The system shall allow a customer to cancel an eligible order.
FR-ORD-5	The system shall allow an administrator to update the status of an order (e.g. processing, shipped, delivered).

3.5 Reviews and Notifications

ID	Requirement
FR-REV-1	The system shall allow a customer who purchased a product to submit a rating and written review for it.
FR-REV-2	The system shall display aggregated reviews and ratings on a product's detail page.
FR-NOT-1	The system shall generate and display notifications to a customer for relevant account and order events.

3.6 Administration

ID	Requirement
FR-ADM-1	The system shall allow an administrator to create, update, and delete products and categories (CRUD).
FR-ADM-2	The system shall allow an administrator to view and manage customer accounts.
FR-ADM-3	The system shall allow an administrator to view and manage all customer orders.
FR-ADM-4	The system shall provide an administrator dashboard summarising sales and inventory.

4. External Interface Requirements

4.1 User Interfaces

The system shall provide a responsive, browser-based user interface implemented as a React single-page application, comprising a public storefront, an authenticated customer account area, and a role-restricted administrator dashboard.

4.2 API Interfaces

The backend shall expose a RESTful HTTP API, organised into 9 functional modules (over 65 endpoints in total), consumed by the React front end. All API responses shall use a consistent, generically-typed response envelope, ApiResponse<T>, to represent success and error outcomes uniformly.

4.3 Software Interfaces

Interface	Purpose
MongoDB (Spring Data MongoDB)	Persistent storage of all domain entities via 17 repository interfaces.
Cloudinary	External storage and delivery of product images.
JSON Web Tokens (JWT)	Stateless authentication tokens exchanged between client and server.

5. Non-Functional Requirements

5.1 Security

ID	Requirement
NFR-SEC-1	The system shall enforce role-based access control (ADMIN / CUSTOMER) using declarative @PreAuthorize checks on protected endpoints.
NFR-SEC-2	The system shall store all passwords using BCrypt hashing and shall never expose password fields through any API response (enforced via DTO classes such as UserDTO).
NFR-SEC-3	The system shall use stateless authentication based on signed JWTs, validated once per request by a dedicated security filter.

5.2 Reliability & Error Handling

ID	Requirement
NFR-REL-1	The system shall handle errors centrally through a global exception handler (@RestControllerAdvice) that maps each exception type to an appropriate HTTP response.
NFR-REL-2	The system shall distinguish resource-not-found, invalid-credential, validation and access-denied errors, returning a consistent ApiResponse<T> error envelope in every case.

5.3 Maintainability & Extensibility

ID	Requirement
NFR-MNT-1	The system shall separate concerns into Controller, Service and Repository layers so that a layer's internal implementation can change without affecting the layers around it.
NFR-MNT-2	The system shall allow a new implementation of an existing service interface (e.g. a new payment method or notification channel) to be introduced without modifying classes that depend on the interface.

5.4 Performance

ID	Requirement
NFR-PERF-1	The system shall return paginated results for product search and listing endpoints to bound response size and latency.

6. System Design Notes (Object-Oriented Design)

This section documents how the design and implementation constraints of Section 2.5 are satisfied in the codebase, and is intended for developers and code reviewers rather than as a set of testable requirements.

6.1 Encapsulation

Every entity class (User, Product, Order, Cart, etc.) declares all fields as private, hiding internal state from the rest of the application. Controlled access is provided through getters and setters generated by Lombok's @Data annotation — the compiled class still exposes only public accessor methods, never the raw fields. DTO classes such as ProductDTO and UserDTO further reinforce encapsulation at the API boundary: internal fields such as a hashed password are never exposed to the client.

6.2 Abstraction

The service layer defines nine Java interfaces (ProductService, OrderService, CartService, AuthService, ReviewService, WishlistService, CategoryService, UserService, NotificationService) describing what operations are available without exposing how they are implemented. Controllers depend only on these interfaces. Spring Data repository interfaces such as ProductRepository extend MongoRepository<Product, String> and gain full CRUD capability without any implementation code being written by the developer.

6.3 Inheritance

Custom exceptions ResourceNotFoundException and InvalidCredentialsException extend Java's built-in RuntimeException, reusing constructor and stack-trace behaviour. JwtAuthenticationFilter extends Spring's OncePerRequestFilter, inheriting the guarantee that the filter runs exactly once per request. The User entity implements Spring Security's UserDetails interface, inheriting a five-method contract that lets Spring Security treat any User object as an authenticatable principal.

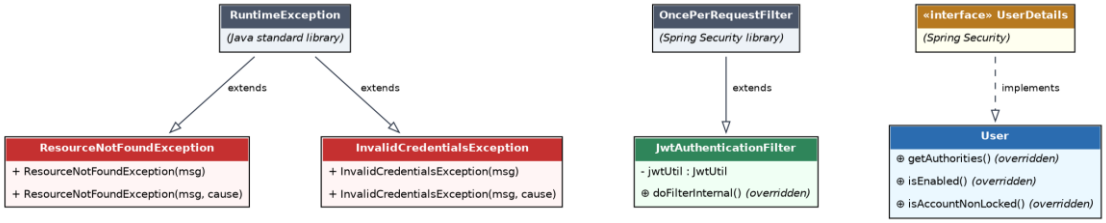
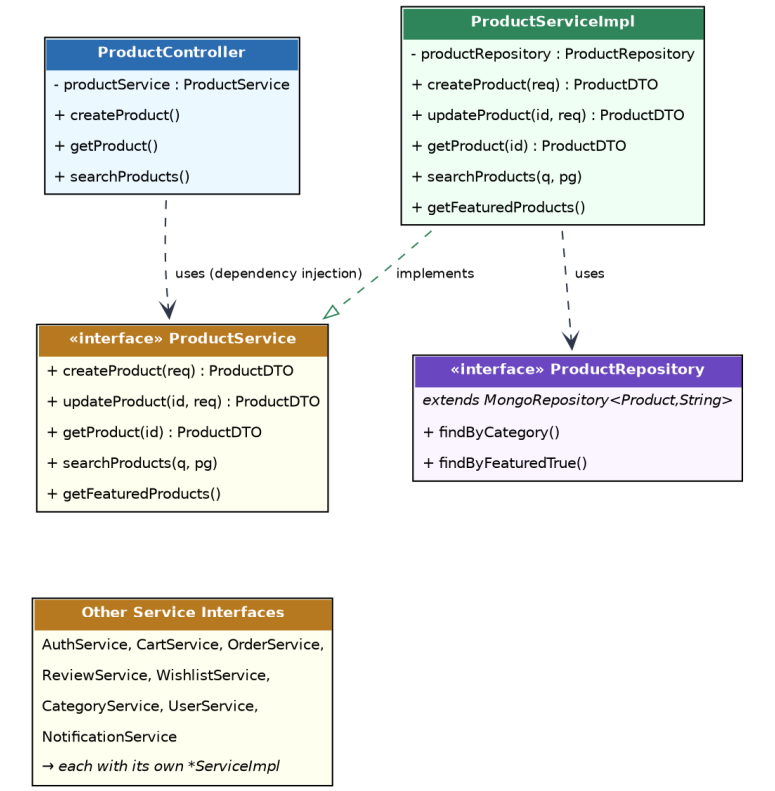


Figure 2: Inheritance hierarchy — custom exceptions, security filter and the UserDetails contract

6.4 Polymorphism

Runtime polymorphism: the User class overrides five methods declared by UserDetails; every *ServiceImpl class overrides the methods declared in its corresponding interface, so a controller holding a reference typed as the service interface invokes whichever concrete implementation is supplied at runtime. The GlobalExceptionHandler dispatches to different @ExceptionHandler methods based on the runtime type of the thrown exception.

Compile-time polymorphism: the generic ApiResponse<T> class overloads its static factory methods success() and error() in multiple forms, with the compiler selecting the correct version based on the arguments supplied.



Abstraction & Polymorphism: Controller depends only on the ProductService abstraction; ProductServiceImpl supplies the concrete behaviour (interchangeable at runtime).

Figure 3: Abstraction and polymorphism in the service layer — the controller depends on the ProductService interface; ProductServiceImpl supplies the concrete, overriding behavior

6.5 Composition and Aggregation

Order is composed of a list of Order.OrderItem objects and a single Order.OrderTracking object; Cart is composed of a list of Cart.CartItem objects; StudyKit is composed of a list of StudyKit.KitProduct objects — in each case implemented as a private static nested class that has no independent existence outside its parent. By contrast, Product relates to Category, Author and Publisher through association/aggregation: a Product stores only the id of its Category, and the Category continues to exist independently if the Product is deleted.

6.6 Core Domain Class Diagram

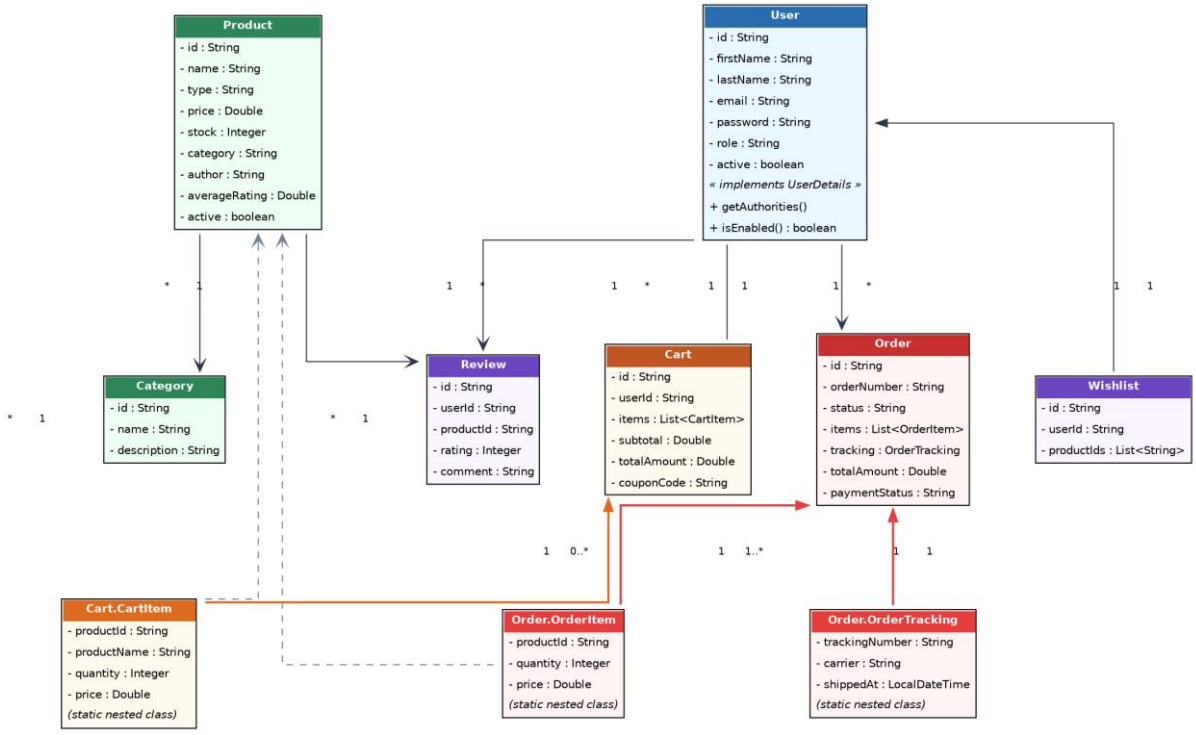


Figure 4: Core domain classes — User, Product, Cart, Order and their relationships

7. Appendix

7.1 Glossary

Term	Definition
DTO	Data Transfer Object — a class used to shape data exposed across the API boundary, distinct from the persisted entity.
JWT	JSON Web Token — a signed token used for stateless authentication.
CRUD	Create, Read, Update, Delete — the basic persistence operations.

7.2 Document History

Version	Description
1.0	Initial Software Requirements Specification, restructured from the project's OOP design report.